APFS FastIndex Manual
A Living Reproduction Textbook

apfs-fastindex project

Living draft, 2026-05-14

Contents

Preface iii

I Problem And Method 1

1 Problem And Scope 2
- 1.1 Project Goal . . . 2
- 1.2 Why APFS Is Not NTFS MFT Walking . . . 2
- 1.3 Current Non-Goals . . . 2

2 Research Method 4
- 2.1 Artifact Types . . . 4
- 2.2 Staged Gates . . . 4
- 2.3 Claim Discipline . . . 5
- 2.4 Source Review Cascade . . . 5
- 2.5 Pinned Citation Style . . . 6

II Raw V1 Pipeline 7

3 V1 Raw Pipeline 8
- 3.1 Pipeline . . . 8
- 3.2 Checkpoint And Scan State . . . 8
- 3.3 OMAP And Object Resolution . . . 8
- 3.4 FS Records . . . 9
  - 3.4.1 Body And Xfield Decoding Rules . . . 9
- 3.5 Namespace And Logical Size . . . 9
- 3.6 Current Open Blocker . . . 10

4 Validation And Oracles 11
- 4.1 Plural Oracles . . . 11
- 4.2 Successful Proof Pattern . . . 11
- 4.3 Cross-Tool Oracle Pairing . . . 12
- 4.4 Validation Gate Cascade . . . 12
- 4.5 Negative Live-State Pattern . . . 12
- 4.6 Inconclusive Pattern . . . 12

III Runtime And Product Boundaries 13

5 Runtime Support 14
5.1 Readable Is Not Supported . . . 14
5.2 Current Allowlist . . . 14
5.3 Fallback Conditions . . . 15

6 Accounting And Product Semantics 16
6.1 Logical Size First . . . 16
6.2 Deferred Metrics . . . 16
6.3 Boot-Root Semantics . . . 16
6.4 Long-Range Roadmap . . . 17

IV Incremental Research 18

7 Identity, Reuse, And Cache 19
7.1 Bare OID Is Not Enough . . . 19
7.2 Subtree Reuse . . . 19
7.3 Persistence And Invalidation . . . 20

V Reproduction 21

8 Implementations 22
8.1 Python Proof Skeleton . . . 22
8.1.1 Current Implementation Boundary . . . 22
8.1.2 Regression Command . . . 22
8.1.3 What Not To Infer . . . 22
8.2 Native Rust Scanner . . . 23
8.2.1 Current Boundary . . . 23
8.2.2 Module Layout . . . 23
8.2.3 Commands . . . 24
8.2.4 Empirical Corrections Distilled Into Code . . . 24
8.3 Native Replacement Path . . . 24

9 Experiment Catalog 26

Glossary 28

Preface

This manual is a living guide to the `apfs-fastindex` research project. Its purpose is to help a future engineer reproduce the work, understand why the scope is narrow, and adapt the method to other projects that need filesystem metadata faster than ordinary high-level traversal APIs can provide.

The source of truth remains the repository artifact trail:

- research synthesis lives in `docs/research/RL-*.md`
- external evidence reviews live in `docs/research/sources/SR-*.md`
- controlled probes live in `docs/research/experiments/EX-*`
- implementation-facing boundaries live in `docs/implementation/`
- the current product intent lives in `spec.md`

This document should not promote hypotheses into design commitments. When it summarizes a claim, it should preserve the project's claim discipline: *Spec* for public documentation, *Observation* for experiment or converging implementation evidence, and *Hypothesis* for plausible but unproven design direction.

The current manual is intentionally skeletal. It records the narrative structure and the canonical cross-links so later research can fill chapters without turning the repository into a second, inconsistent source of truth.

Since the previous revision the project has added a native Rust read-only path (`crates/apfs-fastindex`) covering source gating, descriptor scanning, checkpoint-map validation, container/volume OMAP resolution, volume-superblock decoding under the v1 feature allowlist, and an FS-tree record-family dump. A second cascade of source reviews (`SR-005` through `SR-018`) has tightened checkpoint validation, OMAP lookup semantics, FS-record body layout, xfield cursor rules, fail-closed boundaries, logical-size source precedence, and name normalization. Five new experiments (`EX-10` through `EX-14`) record the empirical state of the native path; `EX-14` is the current open blocker for promoting Rust output past the FS-record family inventory. This revision updates each chapter to track those changes without prematurely claiming work that has not landed.

Part I

Problem And Method

Chapter 1

Problem And Scope

1.1 Project Goal

`apfs-fastindex` is investigating whether an APFS indexer can eventually deliver WizTree-like scans by reading APFS metadata structures directly instead of walking every file through normal filesystem APIs.

The narrow v1 goal is smaller:

- one APFS volume
- one coherent selected state
- namespace reconstruction
- logical size
- fail-closed behavior outside the tested allowlist

The canonical product and v1 scope sources are `spec.md` and `docs/research/contracts/narrow-v1-parser-contract.md`. The long-range route from the narrow parser to a hybrid consumer product is tracked in `docs/research/plans/general-wiztree-for-any-mac-roadmap.md`, which defines staged Gates 0–8 from the narrow Rust full scan to a hybrid product and a parallel R0–R4 release shape.

1.2 Why APFS Is Not NTFS MFT Walking

The project cannot simply copy an NTFS MFT strategy. APFS uses copy-on-write B-trees, object maps, checkpointed transaction state, volume roles, snapshots, clones, sparse files, compression, and modern macOS presentation layers. The research therefore starts with correctness and state selection before speed.

1.3 Current Non-Goals

The current raw v1 path does not claim:

- live startup-disk raw scanning

- merged System/Data/Finder-visible root semantics
- physical, shared, exclusive, or snapshot-retained accounting
- persistent incremental cache reuse
- production performance

Those remain research tracks, not implementation promises.

Chapter 2

Research Method

2.1 Artifact Types

The project uses three durable research artifact types, defined in `docs/research/000-research-index.md`.

RL Research logs. These are living synthesis documents organized by question and decision impact.

SR Source reviews. These summarize external public documentation, open-source implementation behavior, reverse-engineering notes, and support limits.

EX Experiments. These are controlled probes with scripts, generated artifacts, expected outcomes, observed outcomes, and a distilled conclusion.

Raw command output belongs in experiment artifacts. Durable conclusions belong in the experiment README and then in the affected research logs.

2.2 Staged Gates

The staged gates are:

Gate A Minimally correct v1 parser: checkpoint selection, object resolution, FS tree records, namespace, logical size, and validation.

Gate B Support boundary: runtime source classes, encryption, feature gates, and fallback.

Gate C Safe incremental scanning: identity, subtree reuse, persistence, and invalidation.

Gate D Product semantics and optimization: snapshots, firmlinks, volume groups, and performance model.

This ordering prevents performance work from outrunning correctness and prevents product claims from outrunning support evidence.

2.3 Claim Discipline

Every durable claim should be tagged mentally as one of:

- **Spec**: backed by Apple/public documentation.
- **Observation**: backed by repo experiments or converging external implementations.
- **Hypothesis**: plausible, useful, but not yet proven.

The manual should preserve that discipline. It is acceptable for a chapter to say "not yet known"; that is better than encoding a guess as architecture.

2.4 Source Review Cascade

External evidence is consolidated in `docs/research/sources/SR-*.md`. The current set covers:

- `SR-001` v1 support boundary
- `SR-002` checkpoint, OMAP, and root-discovery contract
- `SR-003` FS record taxonomy
- `SR-004` runtime read path and encryption boundary
- `SR-005` checkpoint validation details for the Rust scanner
- `SR-006` OMAP lookup semantics and failure cases
- `SR-007` object-header fail-closed validation
- `SR-008` FS record layout for native v1 parsing
- `SR-009` compression and logical-size precedence
- `SR-010` snapshots, volume groups, and firmlinks
- `SR-011` encryption and runtime read-path boundary
- `SR-012` format drift and feature-bit allowlisting
- `SR-013` checkpoint-map integrity and ephemeral-object validation
- `SR-014` native FS-record body contract
- `SR-015` xfield layout and alignment
- `SR-016` record-body fail-closed boundary
- `SR-017` logical-size source precedence
- `SR-018` name normalization and case behavior

Every `SR-*` note opens with a bottom line, separates Spec from Observation from Hypothesis, and ends with a Decision impact line that names the affected `RL-*` log and the next exact step. Conflicts between Apple's PDF and converged open-source implementations carry an explicit **Conflict Note** so the smallest decisive probe is identifiable from the review alone.

2.5 Pinned Citation Style

External citations should pin both the URL and either a commit SHA or retrieval date. `SR-015` is the current model: the source list cites Apple's reference PDF together with permalinks at fixed commits for `linux-apfs-rw`, `apfs-fuse`, `dissect.apfs`, `go-apfs`, `libfsapfs`, and the Sleuth Kit's APFS module. A claim that three or more independent implementations agree on is treated as stronger evidence than the PDF on its own.

Part II

Raw V1 Pipeline

Chapter 3

V1 Raw Pipeline

3.1 Pipeline

The narrow raw parser pipeline is:

1. source gate
2. checkpoint pinning
3. container root discovery
4. volume root discovery
5. FS record walk
6. namespace entry construction
7. logical-size aggregate construction
8. oracle comparison

The implementation-facing contract is `docs/research/contracts/narrow-v1-parser-contract.md`. The first prototype plan is `docs/research/plans/first-raw-parser-prototype-plan.md`.

3.2 Checkpoint And Scan State

The parser must choose one coherent state and keep all object resolution within that state. `docs/research/RL-01-checkpoint-selection-and-consistency.md` tracks the open checkpoint questions. `docs/research/sources/SR-002-checkpoint-omap-root-contract.md` is the current source review for checkpoint, OMAP, and root discovery.

3.3 OMAP And Object Resolution

Object lookup is not a global `oid -> paddr` map. Resolution needs the owning OMAP context, logical object id, and selected scan state. The active logs are `docs/research/RL-02-omap-and-object-resolution.md` and `docs/research/RL-04-node-identity-cache-keys-and-oid-reuse.md`.

3.4 FS Records

The v1 record surface is intentionally narrow:

- directory records for membership and names
- inode records for identity and metadata
- dstream or equivalent fields for logical size
- symlink xattrs for symlink target fidelity
- hard-link records where present

The current source-backed taxonomy is `docs/research/sources/SR-003-fs-record-taxonomy.md`; the rolling synthesis is `docs/research/RL-03-fs-tree-topology-and-required-records.md`.

3.4.1 Body And Xfield Decoding Rules

FS-record body decoding adds three further source-backed contracts:

- `docs/research/sources/SR-008-fs-record-layout-v1.md` fixes the on-disk layout of `j_drec_*`, `j_inode_*`, `j_dstream_*`, `j_xattr_*`, and `j_sibling_*`.
- `docs/research/sources/SR-014-native-fs-record-body-contract.md` ties body decoding to the validated checkpoint/OMAP/root context.
- `docs/research/sources/SR-015-xfield-layout-and-alignment.md` pins one deterministic xfield cursor: the value area starts immediately after the metadata table and each value advances by `round_up(x_size, 8)`; `xf_used_data` is the padded value-area byte count.
- `docs/research/sources/SR-016-record-body-fail-closed-boundary.md` enumerates the malformed-body hard stops (short fixed bodies, malformed names, duplicate or out-of-bounds xfields, invalid xattr forms, `drec`/`inode` type mismatches, missing sibling mappings).

3.5 Namespace And Logical Size

Namespace output must preserve path identity and file identity separately. Logical size is the only v1 size metric. Physical, shared, exclusive, and snapshot-retained accounting are separate modes.

The logical-size source precedence is fixed by `docs/research/sources/SR-017-logical-size-source-precedence.md`: ordinary, sparse, cloned, hard-linked, symlink, and compressed cases each name the candidate fields (`j_dstream_t.size`, `INO_EXT_TYPE_DSTREAM`, `INO_EXT_TYPE_SPARSE_BYTES`, decmpfs xattr) and the fail-closed condition when they disagree.

Name normalization, case behavior, and APFS hash collisions follow `docs/research/sources/SR-018-name-normalization-and-case.md`. Until the SR-018 fixture lands, stored UTF-8 names are emitted verbatim and lookup-by-name is not claimed.

Related logs:

- `docs/research/RL-06-namespace-reconstruction.md`
- `docs/research/RL-07-size-and-space-accounting.md`

3.6 Current Open Blocker

`docs/research/experiments/EX-14-xfield-layout-variant/README.md` returned `oracle_inconclusive` on a denser EX-13-style fixture: the Rust context provider found four valid checkpoint candidates with highest XID 20 but aborted with `APFS object validation failed: checksum mismatch at block 1031` before publishing `selected_checkpoint`. The rolling roadmap `docs/research/plans/general-wiztree-for-any-mac-roadmap.md` treats this as the gating defect for Rust FS-record body decoding and downstream namespace emission. The next exact step is a focused checkpoint/OMAP-context replay for the EX-14 fixture shape that distinguishes stale checkpoint selection, a missing checkpoint-map/data-ring rule, a Rust validation bug, or a malformed-source hard stop. Until that experiment closes, the manual records the native pipeline as halted at FS-record family inventory.

Chapter 4

Validation And Oracles

4.1 Plural Oracles

There is no single oracle for APFS indexing. The correct oracle depends on the metric and product mode:

- namespace: POSIX/API traversal of the same selected volume or stable view
- logical size: public file metadata APIs and raw inode/dstream comparison
- allocated size: metric-specific public tool and raw extent evidence
- incremental correctness: fresh full raw parse of the same selected state
- boot-root semantics: user-visible macOS namespace only when that mode is explicitly requested

The canonical oracle policy is in `docs/research/RL-10-validation-corpus-and-oracle.md`.

4.2 Successful Proof Pattern

The successful v1 proof pattern is:

1. mount a lab image
2. build a deterministic corpus
3. capture mounted oracle
4. detach the image to stabilize raw state
5. reattach without mounting
6. raw-walk the APFS container
7. normalize and compare path, type, file identity, logical size, and symlink target

This pattern was established in `docs/research/experiments/EX-03-pinned-state-raw-vs-oracle/README.md` and expanded in `docs/research/experiments/EX-04-expanded-pinned-corpus/README.md`.

4.3 Cross-Tool Oracle Pairing

`docs/research/experiments/EX-12-omap-lookup-contract/README.md` established a paired oracle pattern for the Rust path: a probe builds a fresh proof fixture, runs the native Rust scanner and `go-apfs identitydump` against the same `/dev/rdiskN`, replays `obj_phys_t` validation at every Rust-returned `paddr`, re-runs the SR-006 lower-bound rule on Rust's published OMAP samples, and confirms cross-tool agreement on `root_tree.oid`. The `(paddr, object_xid)` divergence between the Rust selected XID and the apparent `go-apfs` active state is recorded as a `go_apfs_active_state_observation` caveat rather than a contradiction.

4.4 Validation Gate Cascade

The current source-backed validation gates that precede any namespace claim:

- `docs/research/sources/SR-005-checkpoint-validation-details.md` for the checkpoint-superblock validation contract.
- `docs/research/sources/SR-006-omap-lookup-semantics.md` for `(omap, oid, max_xid)` lower-bound semantics and OMAP failure cases.
- `docs/research/sources/SR-007-object-header-fail-closed-validation.md` for the single `validate_object_block` chokepoint used by every reader in `crates/apfs-fastindex`.
- `docs/research/sources/SR-013-checkpoint-map-integrity.md` for checkpoint-map ring validation and ephemeral-object handling.
- `docs/research/sources/SR-016-record-body-fail-closed-boundary.md` for malformed-body hard stops before any row is emitted.

4.5 Negative Live-State Pattern

`docs/research/experiments/EX-05-live-pinned-churn/README.md` is the counterexample pattern. A mounted image can be raw-readable while latest-state raw output still lacks a named stable oracle. That result prevents the manual from treating live latest raw traversal as correctness evidence.

4.6 Inconclusive Pattern

`docs/research/experiments/EX-14-xfield-layout-variant/README.md` is the counterexample pattern for an upstream failure: the experiment captured a mounted oracle, ran the Rust scanner, and recorded that the native context provider failed before xfield comparison could occur. Recording the `validation_gaps` list and the failing block address is itself the deliverable; the experiment does not promote a Hypothesis past the available evidence.

Part III

Runtime And Product Boundaries

Chapter 5

Runtime Support

5.1 Readable Is Not Supported

The read-path research separates four claims:

Readable Raw bytes can be opened and sampled.

Parsable Checkpoint/root discovery and raw traversal can run.

Validatable Output can be compared to a named oracle for the requested mode.

Supported The source is inside the product allowlist for that mode.

This vocabulary comes from `docs/research/experiments/EX-08-read-path-matrix/README.md`. The first safe-host execution of EX-08 supports detached unencrypted image-backed APFS for narrow v1 proof work, keeps mounted images `readable_not_supported`, and records startup raw read as `blocked_privilege`.

5.2 Current Allowlist

The current raw-mode allowlist is narrow:

- detached, unencrypted, image-backed APFS containers that match the tested layout
- caller-supplied raw APFS container devices under `/dev/disk*` or `/dev/rdisk*` matching the tested layout
- explicitly stable APFS sources where one scan state can be selected and held for the full run
- mounted lab images only for documented experiments, not product support

The support-boundary source reviews are `docs/research/sources/SR-004-runtime-read-path-and-encryption.md`, `docs/research/sources/SR-011-encryption-runtime-read-path.md` for the encryption boundary, and `docs/research/sources/SR-012-format-drift-feature-allowlist.md` for the incompatible-feature allowlist enforced by the Rust container and volume decoders.

5.3 Fallback Conditions

Fallback or hard-stop conditions include:

- ambiguous or malformed checkpoint state
- unsupported checkpoint layout
- unsupported incompatible feature bits
- unsupported encryption or hardware-backed key requirements
- Fusion or multi-device storage requirements
- requested merged-root semantics
- snapshot/sealed-volume semantics outside the target mode
- no validated corpus for the source class

The rolling logs are `docs/research/RL-08-live-volume-encryption-and-read-path.md` and `docs/research/RL-13-format-drift-compatibility-and-fallback.md`.

Chapter 6

Accounting And Product Semantics

6.1 Logical Size First

The v1 metric is logical size. This is the only size metric with enough current evidence for the narrow raw parser contract. Directory aggregates use unique-inode logical totals within the aggregate root so hard links do not double-count inside one aggregate.

Source precedence for logical size is fixed by `docs/research/sources/SR-009-compression-logical-size-precedence.md` and the wider, scoped table in `docs/research/sources/SR-017-logical-size-source-precedence.md`. The current accounting log is `docs/research/RL-07-size-and-space-accounting.md`.

6.2 Deferred Metrics

The following metrics remain research topics:

- allocated size
- exclusive size
- shared size
- compression storage savings
- snapshot-retained bytes

`docs/research/experiments/EX-09-accounting-probe/README.md` defines the next accounting probe. It explicitly avoids treating `du`, `st_blocks`, raw extents, and Finder-like values as interchangeable oracles.

6.3 Boot-Root Semantics

Raw single-volume namespace and user-visible macOS root namespace are distinct. Modern macOS may involve System/Data volume groups, firmlinks, snapshots, and signed system volume behavior. The manual should never imply that a raw single-volume walk is the same as Finder-visible `/`.

The product-semantics log is `docs/research/RL-11-snapshots-volume-groups-and-firmlinks.md` and the boundary source review is `docs/research/sources/SR-010-snapshots-volume-groups-firmlinks.md`.

6.4 Long-Range Roadmap

The general WizTree-for-any-Mac product path is tracked in `docs/research/plans/general-wiztree-for-any-mac-roadmap.md`. It defines staged gates from the narrow Rust full scan to a hybrid consumer product:

Gate 0 QA discipline (artifacts, oracles, fail-closed checks).

Gate 1 Native narrow Rust full scan (`raw_single_volume_logical_size`).

Gate 2 Local single-volume backend with raw fast path and supported-API fallback.

Gate 3 Finder-visible macOS namespace mode (`os_visible_namespace`).

Gate 4 Encryption and live runtime support.

Gate 5 Size semantics beyond logical size.

Gate 6 Incremental scanning and persistent cache.

Gate 7 Performance engineering.

Gate 8 Product packaging and UX.

A parallel release shape (R0 research harness, R1 narrow Rust MWP, R2 hybrid single-volume scanner, R3 macOS visible namespace scanner, R4 fast repeat-scan product) keeps the audience and the support boundary explicit at every release. The roadmap's blocker register names `EX-14`, the SR-015 xfield replay, the Rust body-field dump, the compressed logical-size fixture, and the live-startup support questions as the current open work.

Part IV

Incremental Research

Chapter 7

Identity, Reuse, And Cache

7.1 Bare OID Is Not Enough

`docs/research/experiments/EX-06-identity-tracking/README.md` showed that the FS root tree logical OID stayed stable while physical address, object XID, checksum, and block hash changed across mutations. Therefore a cache keyed only by OID is unsafe.

The relevant synthesis log is `docs/research/RL-04-node-identity-cache-keys-and-oid-reuse.md`.

7.2 Subtree Reuse

`docs/research/experiments/EX-07-subtree-reuse/README.md` found zero false reuse for exact node-identity matches in a detached image-backed lab corpus. That result keeps subtree summary reuse alive as a future architecture track.

It is not yet a production cache theorem. A future cache entry must include at least:

- source or volume identity
- parser version
- summary schema version
- OMAP domain
- OID
- object XID
- physical address
- checksum or content hash
- expected type/subtype

7.3 Persistence And Invalidation

Persistent cache storage is intentionally deferred. The project must first replace proof-backend raw walking with native summaries and then validate incremental output against a fresh full raw parse.

Related logs:

- `docs/research/RL-05-subtree-reuse-correctness.md`
- `docs/research/RL-09-cache-persistence-and-invalidation.md`
- `docs/research/RL-12-performance-model-and-optimization.md`

Part V

Reproduction

Chapter 8

Implementations

The repository carries two runnable implementations: a Python proof skeleton that remains the authoritative oracle for namespace and logical-size parity, and a native Rust read-only path that has been extended to FS-tree record-family inventory. This chapter documents both and the discipline that keeps them honest about what they prove.

8.1 Python Proof Skeleton

8.1.1 Current Implementation Boundary

The proof-backed parser skeleton is documented in `docs/implementation/narrow-v1-proof-parser-skeleton.md`.

The execution path is:

1. source gate
2. scan-state summary
3. proof raw-walk backend
4. namespace entry normalization
5. directory aggregate construction
6. oracle diff

8.1.2 Regression Command

From the repository root:

```
PYTHONPATH=src python3 -m apfs_fastindex.poc_regression
```

The regression creates a temporary APFS image, builds a small corpus, captures a mounted oracle, detaches the image, runs the proof parser, and compares output.

8.1.3 What Not To Infer

Do not infer native parser performance from the proof skeleton. The proof backend currently shells out through `go run` and exists to keep correctness experiments runnable while native root resolution and FS-record parsing are split out.

8.2 Native Rust Scanner

8.2.1 Current Boundary

The native Rust crate is `crates/apfs-fastindex` and its implementation-facing spec is `docs/implementation/rust-checkpoint-scanner.md`. The crate is the v1 implementation slice; it covers the full read-only path from source-gate to FS-tree record-family inventory but does not yet decode FS-record bodies, emit `NamespaceEntry` rows, compute logical sizes, or claim live-mounted-source correctness.

What the Rust path currently proves on the proof fixture:

- Allowlisted input is a detached APFS `.dmg` image attached with `hdiutil attach -plist -nomount`, or a caller-supplied raw APFS container path under `/dev/disk*` or `/dev/rdisk*`.
- Block zero is validated as an APFS container locator (block size, `NXSB` magic, object type, object id, Fletcher-64 checksum).
- Contiguous checkpoint descriptor areas are scanned for checksum-valid `NXSB` candidates and the highest candidate XID is selected.
- The selected NX superblock is decoded end-to-end and the v1 incompatible-feature allowlist is enforced.
- The per-checkpoint `checkpoint_map_phys_t` ring is walked, ephemeral OID mappings are reported, and `CHECKPOINT_MAP_LAST` is required.
- The container OMAP is opened and queried with the `(oid, max_xid)` lower-bound semantics required by SR-006, with hard stops on the encryption, no-header, and unknown-flag conditions.
- Every reachable volume superblock is decoded and gated by the v1 allowlist (encryption, sealed volumes, normalization sensitivity, unknown incompatible features all force `Unsupported(reason)`).
- Supported volumes have their volume OMAP opened and the FS-tree root validated through it.
- The FS-tree is walked read-only and an FS-record family dump is emitted (`inode`, `xattr`, `sibling_link`, `dstream_id`, `file_extent`, `dir_rec`, `sibling_map`, snapshot families, and an `unsupported` bucket).

8.2.2 Module Layout

```
crates/apfs-fastindex/src/
  lib.rs          orchestration, parser shapes, source gate
  main.rs         apfs-fastindex-scan binary (JSON output)
  block_io.rs     block reads, le_u*, Fletcher-64, test helpers
  object.rs       obj_phys_t validation (SR-007 single chokepoint)
  btree.rs        read-only B-tree node reader (fixed and variable kv)
  omap.rs         OMAP resolver: (oid, max_xid) lower-bound + summary
  container.rs    NXSB decode + checkpoint map ring walk
  volume.rs       APSB decode + v1 feature allowlist (SR-012)
  fs_records.rs   FS-tree record-family dumper (SR-008)
```

The fail-closed contract is centralized in `object::validate_object_block`: checksum, expected object type, expected storage class, encryption flag, no-header flag, optional `oid == paddr` check, and optional `xid <= scan_xid` guard. Every reader in the crate validates a block through this function before trusting its body.

8.2.3 Commands

Run unit tests:

```
cargo test -p apfs-fastindex
```

Build and run the scanner against a detached APFS `.dmg`:

```
cargo run --bin apfs-fastindex-scan -- /path/to/detached-apfs.dmg
```

Run the EX-10 probe (also asserts on the native dump structure):

```
python3 docs/research/experiments/EX-10-rust-checkpoint-scanner/artifacts/probe_ex10.py
```

Run the EX-12 OMAP lookup contract probe:

```
python3 docs/research/experiments/EX-12-omap-lookup-contract/artifacts/probe_ex12.py
```

8.2.4 Empirical Corrections Distilled Into Code

Three empirical corrections from running the Rust path against the real proof fixture have been landed in the crate and recorded in RL-01, RL-02, and RL-03:

1. `OBJECT_TYPE_CHECKPOINT_MAP` blocks carry `OBJ_PHYSICAL`, not `OBJ_EPHEMERAL`, so the validator for the checkpoint-map ring expects `ExpectedStorage::Physical`.
2. B-tree TOC `v_off` measures the distance from the end of the value area to the start of the value; values run forward by `fixed_value_size` or `v_len`. The earlier "value extends backward from `v_off`" assumption mis-decoded OMAP values as `flags=0x10ffff`.
3. The FS-tree root is reached through the volume OMAP, so on disk it carries a virtual `obj_phys_t` (storage flags 0). The FS-tree validator therefore uses `ExpectedStorage::Virtual` and checks `header.oid == volume_omap_lookup.oid` rather than requiring `o_oid == paddr`.

8.3 Native Replacement Path

Remaining native work, in order, with each step landing as its own `EX-*` note before promotion:

1. Resolve the `EX-14` block-1031 context blocker before any new body decoder is added.
2. Replay `EX-13` with the SR-015 source-backed xfield cursor rule and `xf_used_data` validation.
3. Add synthetic negative record-body cases from SR-016.
4. Decode `j_drec_*_key_t` and `j_drec_val_t` to reconstruct the path table without yet emitting `NamespaceEntry` rows.

5. Decode `j_inode_val_t` flags, link count, and embedded fields needed for logical-size selection per SR-009 and SR-017.
6. Decode `j_dstream_id_val_t` and `j_dstream_t` for logical size, and resolve the symlink-target `XATTR`.
7. Resolve `j_sibling_link_val_t` and `j_sibling_map_val_t` for hard-link unification.
8. Wire the resolved namespace into the existing `ParserOutput` / `NamespaceEntry` / `DirectoryAggregate` / oracle-diff contract so the Rust path can be diffed against the Python proof oracle.
9. Re-run the existing Python proof fixture and the expanded EX-04 corpus after each replacement stage. Native parity is not declared until the regression suite passes.

Any support broadening must first update the related `SR-*`, `EX-*`, and `RL-*` artifacts.

Chapter 9

Experiment Catalog

Experiment	Purpose
EX-01	Live checkpoint consistency and runtime boundary. Shows that mounted lab images are useful probes but latest checkpoint moves under writes.
EX-02	Required-record taxonomy and namespace/size corpus design.
EX-03	First pinned-state raw-vs-oracle proof loop.
EX-04	Expanded pinned corpus across case-insensitive and case-sensitive image-backed APFS.
EX-05	Live mounted churn probe. Shows current latest-state raw walker does not validate against baseline or final mounted oracles.
EX-06	OID, paddr, XID, checksum, and block-hash identity tracking.
EX-07	Subtree reuse proof probe for exact node-identity summary reuse.
EX-08	Read-path support matrix. First safe-host execution supports detached unencrypted image-backed APFS for narrow v1 proof work, keeps mounted images `readable_not_supported`, and records startup raw read as `blocked_privilege`.
EX-09	Accounting probe design for logical, allocated, shared, exclusive, compression, and snapshot-retained semantics.
EX-10	Native Rust checkpoint scanner. Source gating, descriptor scan, NX superblock decode, checkpoint-map validation, container/volume OMAP `(oid, max_xid)` resolution, volume superblock decode under the v1 allowlist, FS-tree root validation, and FS-record family dump.
EX-11	Checkpoint-map integrity. Validates the checkpoint-map chain and mapped ephemeral objects on a generated proof fixture and on synthetic malformed checkpoint-map hard-stop cases before native OMAP/root traversal.
EX-12	OMAP lookup contract. Self-paired probe builds a fresh proof fixture, runs the native Rust scanner and `go-apfs identitydump` against the same `/dev/rdiskN`, replays `obj_phys_t` validation at every Rust-returned `paddr`, re-runs SR-006 lower-bound on Rust's published OMAP samples, and confirms cross-tool agreement on `root_tree.oid`. Verdict: `validated_omap_lookup_contract` for the proof fixture.

EX-13	Native FS-record body oracle (Python-first). Decoded DIR_REC, INODE, XATTR, SIBLING_LINK, SIBLING_MAP, and dstream candidates; reconstructed all mounted paths; and selected the blob-relative xfield data alignment matching j_dstream_t.size and INO_EXT_TYPE_SPARSE_BYTES. Verdict: validated_native_record_body_contract; four non-row-critical records still have top-score xfield-layout ambiguity, motivating the fixture-variant successor.
EX-14	Xfield layout variant. Returned oracle_inconclusive: Rust scanner found four valid checkpoint candidates with highest XID 20 but aborted on APFS object validation failed: checksum mismatch at block 1031 before publishing selected_checkpoint, blocking xfield comparison and Rust FS-record body decoding. Treated as a checkpoint/OMAP-context blocker rather than an FS-record-body finding.

Each experiment's durable conclusion is in docs/research/experiments/EX-*/README.md. Generated JSON and command outputs belong under the corresponding artifacts/ directory. Negative and inconclusive experiments are catalog entries on the same footing as positive ones.

Glossary

APFS Apple File System.

Checkpoint A transaction state used to locate coherent APFS metadata.

XID Transaction identifier. Raw scans must avoid mixing incompatible transaction contexts.

OID Object identifier. A logical object id is not by itself a safe parsed-content cache key.

OMAP Object map. The B-tree used to resolve APFS virtual objects to physical locations in the correct domain and transaction context.

FS tree File-system B-tree containing records used to reconstruct namespace and file metadata.

DIR_REC Directory record used for directory membership and names.

INODE Inode record used for file identity and metadata.

Dstream Data stream metadata that carries logical size information for ordinary file content.

Oracle A feature-specific validation source. There is no single oracle for all APFS product semantics.

Raw-readable Bytes can be opened and sampled from a source.

Raw-supported Raw output is validated for the requested product mode and source class.

Logical size The v1 size metric, corresponding to the user-visible file length rather than allocated/shared storage attribution.

Snapshot-retained bytes Storage retained by APFS snapshot semantics. This is not part of raw v1 logical-size output.

Firmlink macOS presentation mechanism that contributes to merged System/Data namespace behavior.

NX / NXSB APFS container superblock (`nx_superblock_t`). The block-zero copy is the locator; the authoritative selected NXSB lives in the checkpoint descriptor area at the highest valid checkpoint XID.

APSB APFS volume superblock. Decoded by the Rust crate under the v1 feature allowlist; encryption, sealed volumes, and unknown incompatible features force `Unsupported(reason)`.

Checkpoint map The per-checkpoint `checkpoint_map_phys_t` ring that maps ephemeral object IDs to physical addresses for the selected transaction; the last block in the ring must carry `CHECKPOINT_MAP_LAST`.

Scan XID The single chosen transaction identifier the parser pins for the duration of a run; all object resolution must be performed against this scan XID.

Xfield The variable-length extended-field area in inode and directory-record bodies. SR-015 fixes one source-backed cursor rule: values start immediately after the metadata table and each value advances by `round_up(x_size, 8)`; `xf_used_data` is the padded value-area byte count.

Sibling link / sibling map The hard-link records (`j_sibling_link_val_t`, `j_sibling_map_val_t`) that unify multiple paths sharing a single file identity.

decmpfs The Apple compression xattr family used by the HFS-compression mechanism on APFS. SR-009 and SR-017 specify when its uncompressed size takes precedence over `j_dstream_t.size`.

Sparse bytes The `INO_EXT_TYPE_SPARSE_BYTES` xfield that records bytes saved by sparse-file allocation; relevant to the size precedence table when the dstream size includes sparse holes.

Fail-closed The behavioral discipline that turns any unrecognized, malformed, or out-of-allowlist condition into a typed hard stop rather than a best-effort guess; centralized in `object::validate_object_block` for object headers and in SR-016 for record bodies.